Compiling for Distributed Memory: A Historical Survey and Taxonomy from Global Arrays to Mesh-Sharded AI Systems

Sameeran Joshi

***Abstract*—Compilers that target machines with physically distributed memories must decide where data lives, where computation runs, and what communication connects the two. These decisions were posed as compiler problems in the late 1980s for hypercubes and multicomputers, standardised in High Performance Fortran in the 1990s, largely left to hand-written message passing in the 2000s, and have returned in the 2020s as sharding propagation for accelerator meshes and wafer-scale machines. This article surveys that history with a reproducible method: an operational definition of a distributed-memory compiler, an openly maintained and schema-validated catalog, and a taxonomy on independent axes that separates the decision procedure for decomposition (directives, closed-form analysis, analytic optimisation, or profiling and search) from the representation of ownership and from the communication family. Coding the 54 catalog papers whose full text was read, we test seven generalizations across the eras. A run-time stage accompanies data-dependent ownership in every read system that describes it. The cost models moved from analytic network models in the 1990s to measured or simulated programs from 2019. The leading optimisation objective moved from message count to memory fit and productivity. Block-cyclic and mapping-array decompositions disappear after about 2000. Pure element-level communication disappears after the 2000s, and owner-computes survives in kernel compilers while sharding systems replace it with propagation. We state the evidence strength and the refuting observation for each generalization, and close with the open problems they leave, including the comparison of measured and modelled cost that no read paper makes.**

***Index Terms*—Distributed-memory compilers, data distribution, owner-computes, communication generation, High Performance Fortran, inspector–executor, polyhedral compilation, SPMD partitioning, tensor sharding, wafer-scale compilation.**

I. Introduction

A machine whose memory is distributed forces a compiler to answer three questions that a shared-memory compiler can defer to hardware: where each datum is stored, which place executes each operation, and what messages must flow so that operands are present when they are needed. Compilers that address these questions for hypercube and multicomputer targets appear in the 1988 literature [1]–[3]. Within five years the answers had been standardised in the directive language High Performance Fortran (HPF) [4], and the message-passing interface MPI had been standardised as the portable alternative in which the programmer answers all three questions by hand [5]. We observe that the same three questions are now being answered again, by SPMD partitioners such as GSPMD [6] and PartIR [7], by distributed tensor-algebra compilers such as DISTAL [8], and by compilers for wafer-scale and spatial accelerators [9], often with terminology that does not acknowledge the earlier work.

Manuscript draft generated from the open catalog at https://github.com/Sameeranjoshi/dist_compilers. This is a working draft; see the repository ROADMAP for status.

A. Why the problem keeps reappearing

Every time a new class of machine with distinct memories becomes important—hypercubes, clusters, multi-GPU nodes, accelerator pods, wafers—the trade-off between programmer control and compiler automation is re-opened, and the compiler community rebuilds a stack of ownership representation, computation mapping, and communication synthesis for that class. The repository that accompanies this survey exists to make that rebuilding less amnesiac: it records, with evidence, which mechanism solved which problem, under what assumptions, and with what limitation.

B. Scope

We define a *distributed-memory compiler* as a compiler, compiler framework, programming system, or compiler-integrated runtime that transforms a program for execution across multiple computational places whose memories are physically or logically distinct at the relevant compilation level (Section II gives the inclusion criteria). Multiple GPUs are a distributed-memory target when the compiler must partition data or computation and generate communication between device memories; wafer-scale and spatial architectures count when individual processing elements have distinct local memories and data movement must be compiled explicitly. Distributed databases, distributed operating systems, cloud schedulers, hardware-only papers, and shared-memory-only compilers are out of scope.

C. Central question and research questions

The central question is: *how have compilers for machines with physically distributed memories represented, decomposed, mapped, communicated, optimized, and executed data and computation?* We refine it into ten research questions (RQ1–RQ10) on programming abstraction, data decomposition and ownership, computation decomposition, communication generation, the compiler/runtime boundary, compiler representation, optimization objectives, evaluation, historical continuity, and open problems. Section V maps each to an axis of the taxonomy.

D. Contributions

1) An operational definition and inclusion criteria for distributed-memory compilers that span 1950–2026 while keeping pre-1980 work explicitly labelled as *foundations* rather than as distributed-memory compilation (Section III).
2) A reproducible survey methodology with an open, schema-validated catalog in which every entry carries evidence pointers and a screening status (Section II).
3) A common architecture for distributed-memory compilers (Section IV) and a multi-axis taxonomy (Section V), both derived from the catalog.
4) A historical narrative organised around six cross-era findings, each backed by a comparison table generated from the catalog (Section VI), and mechanism sections that place each system on the taxonomy's axes (Sections VII–XI).
5) Seven generalizations across the eras, each stated with its evidence strength and the observation that would refute it, and a set of open problems they leave (Sections XII and XIII).

[TODO: We do not claim that this is the first comprehensive survey of the area; an explicit search for prior surveys is logged in the repository and the claim will be revisited once it is complete. Known prior overviews include [10]–[12].]

II. Survey Methodology

A. Scope and inclusion criteria

A system is included when it performs or represents at least one of: data distribution or ownership; computation partitioning or placement; communication generation; collective insertion; halo or ghost-region construction; global-to-local address conversion; import/export schedule generation; redistribution or resharding; hierarchical parallelism across nodes or devices; distributed execution-plan generation; or distributed-memory performance modelling used by a compiler. We classify included work as *central* (automatic distributed-memory parallelization; global-array and data-parallel compilers; Fortran D, HPF and relatives; automatic message-passing generation; distributed dense and sparse tensor compilers; SPMD partitioners; distributed GPU/accelerator compilers; spatial and wafer-scale compilers) or *bridge* (languages whose compilers implement distributed memory; compiler-integrated runtimes such as inspector–executor systems; PGAS compilers; task/region systems where placement is central; distributed-training systems with substantial plan generation; performance models that drive compiler decisions). Excluded are distributed databases and operating systems, generic job scheduling, routing protocols, hardware-only papers, application scaling studies, shared-memory-only compilers, single-GPU kernel compilers, and pure runtimes without a compiler component. A small number of architecture papers are retained in the catalog solely to date target classes; they are not classified on the compiler axes.

B. Sources and queries

Bibliographic metadata were taken, in order of preference, from the DOI record (via Crossref), arXiv, official author or institution copies, official project documentation, and trusted bibliographic indexes. No paywall was bypassed; where full text was not available the catalog marks the entry `paywalled` or `abstract-only` and no mechanism is inferred from the title. The initial query set (25 phrases such as "data distribution compiler", "communication set generation", "inspector executor distributed memory", "SPMD partitioner compiler", "wafer scale compiler") is listed in the repository together with a discovery log recording each query, date, hits, and screening decision. [TODO: Report per-database hit counts once the full query sweep over ACM DL and IEEE Xplore is logged.]

C. Snowballing

From every anchor paper we snowball backward (references), forward (citing works, prioritising those that extend, critique, evaluate, or revive the anchor), laterally (same system, project, or authors), across surveys (works cited by several independent surveys are prioritised), and by *gap*: for every empty cell of the taxonomy coverage matrix we search for that combination specifically.

D. Screening and evidence levels

Each catalog entry carries one of five statuses: *discovered* (metadata only), *screened* (abstract and introduction checked), *core* (clearly relevant), *read* (enough read to summarise accurately), and *verified* (metadata, categorisation, and summary reviewed against the primary source). Only *read* and *verified* entries support mechanism-level claims in this article. A paper is called a survey only if it is a survey or an invited overview; for example, Lee's "Techniques for compiling programs on distributed memory multicomputers" [13] is catalogued as a technique paper.

E. Reading strategy and paper cards

For most papers we read the abstract, introduction, system overview, related work, and conclusion; we read the full paper when it introduced a major system, defines a taxonomy category, supports a key historical claim, has experiments that we compare, or supplies a mechanism central to the synthesis. Every core paper receives a paper card in the repository listing problem, input abstraction, target, decomposition, mapping, communication, compiler/runtime split, evaluation, limitation, lineage, and the exact sections used.

F. Evidence labelling

Statements in this article are of three kinds: source-supported facts, which carry a citation; survey taxonomy introduced by us, flagged by "we categorize" or "we group"; and inferences, flagged by "appears", "we observe", or "we hypothesise" and listed with their current evidence in the repository's historical-claims file. Conflicts between sources are recorded rather than resolved silently.

G. Limitations

Full text of many 1988–2000 papers is paywalled; their summaries rely on open author copies where available. Citation counts are not used as a relevance criterion. Dates of era boundaries are provisional and are justified by the dated events listed in the catalog.

III. Foundations (c. 1950–1979)

This section separates what the distributed-memory compilers of the late 1980s inherited from what they invented. Nothing here is a distributed-memory compiler. The machines of the period had a single address space or lock-step SIMD control, and placing data in memories was not a compiler concern. Where a foundations paper has no card in the repository yet, we state only its bibliographic record and mark the content claim as pending.

A. Automatic programming

The FORTRAN automatic coding system established that a translator could generate object code for numerical loops from a higher-level notation [14]. We take this as the first instance of the bargain that every later system renews: the programmer writes a higher-level program, and the compiler makes the low-level decisions. [TODO: Add a card for [14] and quote its code-quality claims only after reading.]

B. Dependence analysis and loop parallelism

Lamport's paper on the parallel execution of DO loops, published in 1974, is the standard reference for reordering nested loop iterations so that independent iterations run together [15]. Kuck and colleagues presented dependence graphs as the representation on which compiler optimisation and parallelisation are built [16], and Allen and Kennedy translated FORTRAN loops into vector form using the same analysis [17]. Distributed-memory compilers reused this dependence representation without change for the iteration and access analysis; what they added was an ownership function. Section VII shows this for the Fortran D and PARADIGM lineage. [TODO: Read [16] and [17] and replace this paragraph with card-backed statements.]

C. Machine abstractions

The ILLIAC IV array processor [18] and the CRAY-1 vector computer [19] are the two regular-parallel machine classes that the early loop compilers targeted. Dennis and Misunas proposed a dataflow processor architecture in 1975 [20], and a 1977 MIT workshop report on dataflow computers and program organisation is in the catalog from a supplied copy [21]. Kung's article on systolic architectures [22] and the Cosmic Cube multicomputer [23] establish the two architectural ancestors of the spatial and distributed targets discussed in Sections XI and VI. [TODO: Read [22]: the supplied copy has no text layer and must be read visually before any content claim is made.]

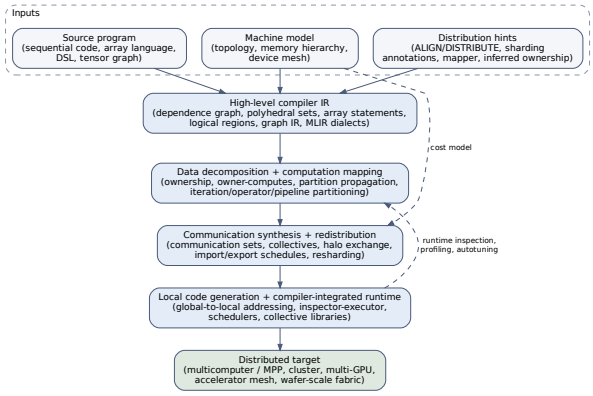


Fig. 1. The stages that recur across the catalog's distributed-memory compilers. Most 1990s systems are linear through the communication stage. The dashed edges are where systems differ on the compiler/runtime boundary (axis F): runtime inspection, profiling and cost models feed back into decomposition. The figure is original to this survey and is abstracted from the cited systems.

D. Data-parallel programming

Hillis and Steele described data-parallel algorithms on the Connection Machine, in which one operation applies to all elements of a data set at once [24]. This is the programming model that Fortran 90 array statements and, through them, HPF inherited. [TODO: Verify the lineage claim against the HPF specification [25] once that record is read.]

E. Synthesis

The foundations supplied three things to the next era: a trusted pipeline for transforming loops, a legality test for reordering them, and machine models with regular parallel structure. What they did not supply was any representation of where data resides. That omission became the central compiler problem of the late 1980s, once machines with private memories appeared [23].

IV. A Common Compiler Architecture

We abstract the systems in the catalog into the pipeline of Fig. 1. The inputs are a program (sequential code, an array language, a domain-specific language, or an operator graph), a machine model (processor arrangement, device mesh, memory hierarchy), and distribution information. That information is either supplied by the programmer, as ALIGN and DISTRIBUTE directives [4], sharding annotations [6], or a task mapper [26], or it is inferred.

a) Intermediate representation: The representation determines which decisions can be made statically. Vienna Fortran's distributions are declarations attached to arrays [27], and Kali's distributions are declared with processor arrays and on-clauses [28]. Koelbel's closed forms are derived from the subscripts of forall loops [29], and Chatterjee et al. give a finite-state representation of local access sequences for cyclic(k) distributions [30]. The polyhedral framework represents iteration and access spaces as integer sets and

was used for MPI generation by Bondhugula [31]. Mesh-TensorFlow names tensor dimensions and maps them onto a processor mesh [32], and MLIR's shard dialect represents grids and per-axis shardings as IR attributes [33].

b) Data decomposition and computation mapping: Kali's doall loops execute an iteration on the processor named by an on-clause, and the report describes owner-based placement [28]. Chen et al. compile Crystal programs, whose data fields carry communication metrics, by reshaping them with morphisms [3]. In GSPMD, sharding annotations are completed by propagation through the graph with priorities, and each operator is then partitioned [6]. Each approach derives a computation mapping from a data mapping, though the direction of derivation and the place where the decision is made differ.

c) Communication synthesis and redistribution: Communication can be derived in closed form from the distribution functions [29], [30], recognised as collectives, or determined at run time from index arrays [34]. Redistribution between distributions is handled by pairwise communication plans in Ramaswamy et al. [35], and by phase-level planning in Palermo et al. [36]. In MLIR's shard dialect, collectives such as all-gather, all-reduce and all-to-all are operations with the participants inferred from the sharding [33].

d) Local code generation and runtime: The PARTI/CHAOS line supplied schedules for off-processor accesses [37], [38]. Legion executes a task graph over logical regions and maps them to memories through a mapper [26]. Rogers and Pingali emitted per-process C code with run-time resolution as a fallback [39]. The amount of work delegated to the runtime is axis F of the taxonomy.

e) Synthesis: The pipeline has been stable since the early 1990s. What has changed is the input abstraction (axis B), the representation on which decomposition is decided (axis K), and which stages are static (axis F). These three differences are the axes along which the eras of Section VI separate.

V. Taxonomy

A chronological list cannot separate the systems this survey is about, because the same mechanism returns under different names in different eras. We therefore classify each paper on independent axes. Fig. 2 shows the axes, the catalog carries eleven, and each entry takes one or more values on each. The value lists are maintained in the repository and enforced by a validator.

A. The axes

The axes answer different questions. Target (axis A) asks what machine the code runs on. Source abstraction (B) asks what the programmer writes. Data decomposition (C) asks how ownership is represented, and computation mapping (D) asks how work follows ownership. Communication synthesis (E) asks what messages or collectives the compiler produces. The compiler/runtime boundary (F) asks which decisions are taken when, and compiler architecture (G) asks how the compiler is built. Optimisation objective (H), maturity (I), paper type (J) and compiler representation (K) complete the set.

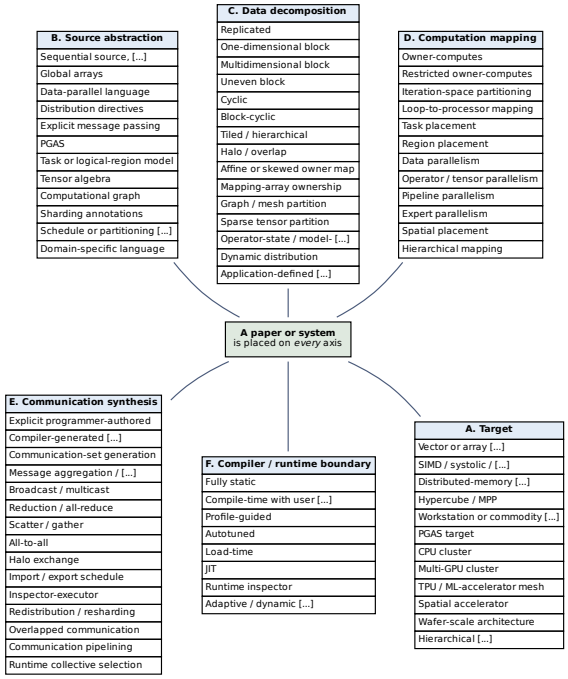


Fig. 2. The taxonomy as orthogonal axes rather than a tree. A paper or system takes one or more values on every axis. Only six axes are drawn here; the catalog carries eleven. The figure is generated from the machine-readable taxonomy so that it cannot drift from the catalog.

B. Refining the compiler/runtime axis

Our first coding of axis F was too coarse. It had one value, "fully static", for every system that decides without running the program. That value merged two different procedures. Kali's closed-form communication sets are obtained by analysis of subscripts. Gupta and Banerjee's distribution is chosen by minimising a weighted cost over constraints [40], and Lee's and Palermo et al.'s layouts come from dynamic programs over a cost model [13], [36]. Kong et al.'s mapping comes from a solver over a volume cost [41]. These are different decision procedures, and they behave differently when the program changes. We therefore split the value into an analytic-optimisation value, and we record directives, profiling and search separately (Table I). The refined axis is the basis for the claims in Sections X and XII.

C. A value the axes did not have

Owner-computes assigns each statement to the owner of its left-hand side. Sharding systems do not assign statements at all: the sharding of a value is propagated through the graph, and each operator is partitioned according to the sharding of its operands [6], [7]. We added a partition-propagation value to axis D so that the two mechanisms can be compared rather than conflated.

D. Coding and its limits

Tags were assigned to each entry from the section of the paper that supports them, and the evidence list of each card names the section. Coding was done by one coder, with agents drafting cards and the author checking tags. A second independent coding is a required step before any count is reported as a finding. The counts in Tables I–IV are over the 54 catalog papers whose full text was read; the era sizes are 9 to 15, so a difference of two or three papers is not a finding.

VI. Historical Evolution

The eras of Fig. 3 are a useful index but not an explanation. This section states what changed between them, using the comparisons of Tables I–IV, and we claim only what those comparisons support. Era 0 and Era 1 have no full-text reads in the catalog, so nothing here is tested for them; the foundations are discussed in Section III from bibliographic records.

A. Finding 1: the runtime stage follows irregular ownership

In the read papers, every system that describes how irregular ownership is resolved also has a runtime stage: an inspector, a run-time partitioner, a dynamic scheduler, or a JIT. This covers 17 papers from the PARTI and CHAOS line [34], [37], [38], [42], [43], from Kali [28], from Legion [26] and Regent [44], from the HPF compiler overviews [10], [11] and from sparse tensor compilers [45], [46]. Three papers were removed from the test after a coding error was found in their irregular tag, and Vienna Fortran's INDIRECT mappings are excluded because the report says run-time techniques are needed but describes no mechanism [27]. The claim we can defend is narrower than "irregularity needs a runtime". It is that a runtime stage is needed when the owner of an element is data-dependent and is not known until the program runs. Where the map is fixed in the source, the compiler can often emit the schedule statically, as the closed-form work of Koelbel shows for affine subscripts [29].

B. Finding 2: the cost model moved from closed-form to measured

Three systems in the read set optimise a distribution over an analytic model: Gupta and Banerjee's constraint method, which the authors report is NP-hard in general [40]; Lee's layout dynamic program [13]; and Palermo et al.'s phase search [36]. Kong et al.'s solver is the one recent analytic system in the set [41]. Table I shows the procedure by era. The systems of 2019 onward that search do so over configurations scored by a simulator or by profiles, not over a closed-form volume model: FlexFlow profiles each operator and re-simulates [47], PipeDream uses a single-GPU profile [48], and Alpa uses profiled stage costs with an integer program [49]. One reading is that measured or simulated costs replace the network model that the 1990s optimisers had to assume. Another is that the search spaces changed. The data cannot separate these, and we flag this as the first open question in Section XII.

TABLE I
Decision procedure for the decomposition, by era. Counts are papers whose full text was read.

Era (papers read)	programmer directives	closed-form analysis	analytic optimisation
Era 2 (1988–92) (n=11)	5	2	1
Era 3 (1993–2000) (n=15)	8	3	2
Era 4 (2000–10) (n=9)	3	3	0
Era 5 (2010–19) (n=9)	1	2	0
Era 6 (2020–26) (n=10)	3	3	1

TABLE II
Optimisation objectives recorded for read papers, by era. A paper may carry several objectives.

Era (papers read)	communication-volume	message-count	load-balance	local-mem
Era 2 (1988–92) (n=11)	4	4	3	
Era 3 (1993–2000) (n=15)	10	11	7	
Era 4 (2000–10) (n=9)	2	4	2	
Era 5 (2010–19) (n=9)	7	3	4	
Era 6 (2020–26) (n=10)	8	1	3	

C. Finding 3: the objective moved from messages to memory and productivity

Message count was the most common recorded objective in Era 3, and it has nearly disappeared from the recent systems (Table II). Local memory fit grows from one paper in Era 2 to eight in Era 6, and productivity grows from none to seven. Search cost appears only in the two most recent eras. The 1990s compilers minimised the number of messages because each message carried a large fixed startup cost on the machines of the time [50]. The recent compilers are judged by whether a model fits in memory and whether a programmer can write it, which is why capacity and productivity dominate their evaluations [6], [7]. This explanation is an inference from the objective counts; the machine-cost argument is supported only by the Fortran D evaluation.

D. Finding 4: the decomposition vocabulary changed

Block-cyclic distributions appear in 6 of 11 papers in Era 2 and 9 of 15 in Era 3, and in none after Era 4. Mapping arrays follow the same course. Model-state sharding appears from Era 5, and sparse-tensor partitions appear only in Era 6 (Table III). The block-cyclic family was the target of the closed-form address arithmetic that made HPF compilation possible [30]. Its disappearance in the recent tensor systems follows from their use of even block and mesh shardings, which need no cyclic address arithmetic. The sample is small, so this finding should be re-tested on a broader set.

E. Finding 5: owner-computes gave way to propagation, but not uniformly

Owner-computes is the recorded rule in 6 of 11 papers in Era 2 and 11 of 15 in Era 3, and it falls to 2 of 9 in Era 5. In Era 6 it is recorded in 6 of 10 papers and partition propagation in 2 (Table IV). The two propagation systems are GSPMD and PartIR [6], [7]; the owner rule remains in the kernel compilers of wafer-scale and tensor work [9], [51]. The owner rule has not been replaced everywhere. It survives

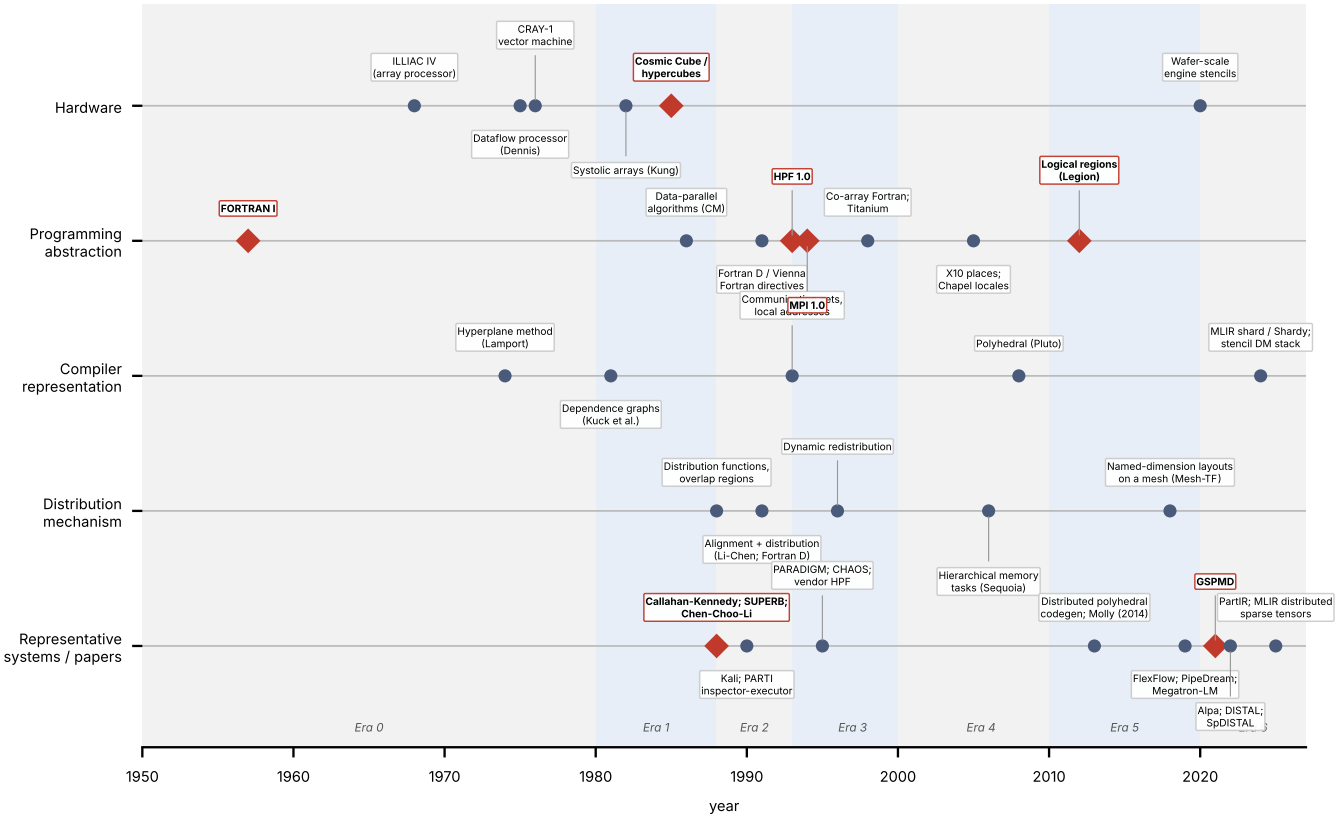


Fig. 3. Timeline 1950–2026 with lanes for hardware, programming abstraction, compiler representation, distribution mechanism, and representative systems. Diamonds mark transitions that open a new era. Each event is tied to a catalog entry; era boundaries are provisional.

TABLE III
DATA-DECOMPOSITION VOCABULARY IN READ PAPERS, BY ERA.

Era (papers read)	block-cyclic	affine-owner-map	halo-overlap	model-state-sharding	sparse-tensor-partition	mapping-array
Era 2 (1988–92) (n=11)	6	2	3	0	0	5
Era 3 (1993–2000) (n=15)	9	2	2	0	0	1
Era 4 (2000–10) (n=9)	3	0	2	0	0	0
Era 5 (2010–19) (n=9)	0	1	2	2	1	0
Era 6 (2020–26) (n=10)	0	0	3	2	2	0

TABLE IV
OWNER-COMPUTES VERSUS PARTITION PROPAGATION IN READ PAPERS, BY ERA.

Era (papers read)	owner-computes	partition-propagation
Era 2 (1988–92) (n=11)	6	0
Era 3 (1993–2000) (n=15)	11	0
Era 4 (2000–10) (n=9)	3	0
Era 5 (2010–19) (n=9)	2	0
Era 6 (2020–26) (n=10)	6	2

where the programmer writes the loop and the compiler places iterations, and propagation is used where the program is a graph of operators and the sharding can be inferred. This split is a claim about the programming model, not about the era, and it is the reason the taxonomy needed a separate propagation value.

TABLE V
COMMUNICATION FAMILY IN READ PAPERS, BY ERA: ELEMENT-LEVEL (SETS, POINT-TO-POINT, INSPECTOR SCHEDULES) VERSUS COLLECTIVE.

Era (papers read)	element-level only	collective only	both
Era 2 (1988–92) (n=11)	4	1	6
Era 3 (1993–2000) (n=15)	4	2	8
Era 4 (2000–10) (n=9)	2	0	4
Era 5 (2010–19) (n=9)	1	1	5
Era 6 (2020–26) (n=10)	0	2	8

F. Finding 6: pure element-level communication is disappearing

Systems whose communication is only element-level (sets, point-to-point sends, inspector schedules) number 4 of 11 in Era 2 and 4 of 15 in Era 3, and fall to 1 in Era 5 and none in Era 6. Most systems now record both kinds (Table V). The pattern suggests that element-level generation survives inside collective-based systems, for example as the local sets of a collective, rather than as the whole strategy. The counts cannot distinguish these; the Era 5 and Era 6 samples are nine and ten papers.

G. What the eras do and do not explain

The era boundaries in Fig. 3 were drawn from dated events. The findings above do not all change at those boundaries. The objective shift appears between Eras 3 and 5, the decompo-

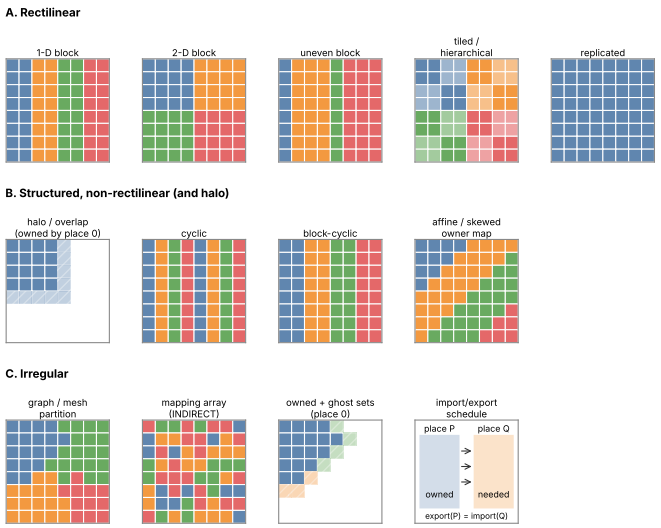


Fig. 4. Data-decomposition classes drawn as example grids coloured by owning place. Rectilinear forms (A) have closed-form owner functions. Structured non-rectilinear forms (B) need block-cyclic address arithmetic. Irregular forms (C) need a mapping array or graph partition and, in the systems surveyed, a run-time stage. Hatched cells are ghost copies. The examples are illustrative.

sition shift between Eras 4 and 5, and the owner-rule shift in Era 6. A chronology therefore explains the timing of each shift less well than the axes explain what shifted.

VII. Data Decomposition and Computation Mapping

A. Three classes of ownership

We group the decompositions in the catalog by what the compiler must know to generate code for them (Fig. 4). The first class is closed-form owner functions: block, cyclic, block-cyclic and uneven block. For these the owner and the local index are arithmetic on the global index, and Chatterjee et al.'s finite-state machine generates the local access sequence for cyclic(k) distributions with affine alignment [30]. The second class is declared maps: mapping arrays, graph partitions and user-defined distribution functions. Vienna Fortran's INDIRECT and user-defined distribution functions are of this kind [27], [52], as are CHAOS's partitioner-driven translation tables [43]. For these the owner of an element is data, so the compiler needs a table and a run-time stage (Section IX). The third class is propagated shardings: a per-dimension assignment of tensor axes to mesh axes, with replication where no axis is named [6], [32]. Here the compiler needs the mesh and the propagation rules, not a table of owners.

The classes predict the decision procedure. Closed-form owners admit exact analysis, so the compiler can decide statically, as in Koelbel's closed forms [29]. Declared maps force a run-time stage. Propagated shardings admit a static procedure over the graph, but the procedure is heuristic when shardings cannot be inferred uniquely [6].

B. What the decomposition vocabulary shows

Table III counts each feature by era, and Finding 4 in Section VI discusses the block-cyclic and mapping-array decline. The one feature present in every era is halo overlap: 3, 2, 2, 2 and 3 of the read papers in Eras 2 to 6 record it. It appears as overlap areas in Fortran D and Vienna Fortran [52], [53], as halo exchange in the dmp dialect [54], and as neighbour exchange on the wafer-scale fabric [9], [51]. Halo overlap is the one ownership feature that is independent of the programming model. Its persistence suggests that neighbour data movement is a property of stencil-like computation rather than of a language era.

C. Assigning computation

The computation rules follow the classes. Under closed-form and declared ownership the rule is owner-computes or an equivalent (Kali's on-clauses [28], Fortran D's owner partitioning [53]). Under propagated shardings the rule is that each operator is partitioned from the shardings of its operands [6]. Task-based systems place tasks and regions through a mapper, which decouples the rule from ownership [26]. Table IV records the counts by era, and Finding 5 in Section VI gives the reading.

D. Alternatives to ownership

Some systems do not decide ownership first. Pipeline systems assign stages of layers to devices and derive the data movement between them [48], and expert parallelism assigns sub-networks to devices [6]. For these the unit of placement is a stage or a sub-network rather than an array element. The catalog records them under computation mapping, and they are the main reason the ownership-based axis does not describe the recent systems completely.

VIII. Communication Generation and Redistribution

Communication is derived in two ways in the catalog. Element-level generation computes, for each reference, which elements move between which places, from the subscripts and the distribution. Collective generation recognises an operator's pattern and emits a collective with participants fixed by the sharding. Most systems in the recent eras use both, and the element-level form survives mainly as the local set that feeds a collective (Finding 6 in Section VI).

A. Element-level generation

Koelbel derived send, receive and iteration sets in closed form for forall loops over block and cyclic arrays with linear subscripts [29]. Chatterjee et al. generate local addresses and communication sets for cyclic(k) distributions [30]. Shen et al. remove iteration variables that do not change which element or processor is involved, reducing generated communication on one LU kernel [55]. Bondhugula's generator computes flow-out and write-out sets per tile and uses run-time helper functions to find the receiving processes [31], [56]. All four derive communication from affine index expressions. Their generality is limited by the affine assumption, and their cost is paid at compile time or in a small run-time helper. This is the classical regime of the catalog.

B. Collective generation

GSPMD inserts collectives where propagated shardings disagree [6], and the MLIR shard dialect exposes all-gather, all-reduce, all-to-all, reduce-scatter and halo update as operations whose participants follow from the sharding [33]. Kong et al. forbid the crossed mappings that would need point-to-point traffic, so only collectives are generated, and each (operator, tensor) communication vector selects a sub-communicator [41]. Vaidya et al. implement blocking get, set and update-slice operations over MPI all-to-all and all-to-allv [46]. Collective generation trades generality for a simpler model. Once the sharding is fixed the participants are known, so the cost of a collective can be stated in volume alone. Kong et al.'s model is volume-only for this reason, and it is the simplest model in the read set [41].

C. Halo exchange and overlap

Halo exchange is the communication pattern that appears in every era (Section VII). Bisbas et al. lower declared halo exchanges to non-blocking sends and receives, and note that each halo is a separate exchange with no overlap and no diagonal exchange [54]. Stawinoga et al. convert halo exchanges into chunked prefetches overlapped with computation on the wafer-scale engine [51]. Rocki et al. overlap receiving with multiplication but keep the all-reduce blocking [9]. The overlap objective is recorded in between one fifth and under half of the read papers in each era (Table II). Overlap is treated as a property of the halo or the chunk, and reductions are the usual exception. Among the read papers we did not find a reduction overlapped with the computation that feeds it; this is a candidate open problem, to be confirmed by a targeted search.

D. Redistribution

Redistribution is the change of distribution between phases. Ramaswamy et al. represent regular distributions as families of line segments, so the data to move between two processor sets is their intersection [35]. Palermo et al. choose phases by a shortest-path search that includes the redistribution cost [36]. In the recent systems redistribution is a resharding between two shardings, inserted where the propagation finds a mismatch [6], [7]. The concept is the same, the input is different: a pair of line-segment families in the 1990s, a pair of mesh shardings now.

IX. Runtime Analysis and Irregular Computation

The boundary between compiler and runtime is where the systems differ most sharply. Finding 1 in Section VI states the central regularity: in the read set, every system that resolves irregular ownership has a runtime stage. This section explains the forms that stage takes and the costs they carry.

A. Three forms of run-time stage

The inspector–executor form separates an inspection of the index arrays from the execution that uses its result. Saltz and colleagues' executor runs a table-driven schedule built by a preprocessor, and the executor overhead is measured at about 9–10% for sparse matrix-vector multiplication on an iPSC/2 [34]. Kali caches the results of its inspector across repeated executions [28]. The reuse condition is the decisive design point. Ponnusamy et al. reuse inspector results with a test that is conservative and can add overhead in some cases [43].

The dynamic-scheduling form moves the decision out of the compiler. Legion moves data-movement decisions to a runtime that reasons about logical regions and privileges [26]. Dathathri et al.'s dataflow runtime evaluates task functions per node [57]. The run-time-resolution form is a fallback. Rogers and Pingali combine run-time resolution, which is correct for every reference, with compile-time resolution where ownership can be analysed [39].

B. What the stage costs

The stage costs an inspector pass or a dynamic scheduler on every change of the index arrays. The 1990s systems amortised this over repeated executions of the same loop [28], [34], and the sparse tensor systems do the same over repeated tensor operations [45]. The adaptive variants accept a schedule rebuild when the indirection changes [38]. We did not find a read paper that reports the break-even point against a fully static alternative for the same program, so the trade-off is described but not measured.

C. Where runtime stages are not needed

Three read papers have no run-time stage for their distributions: Gupta and Banerjee's constraint method [40], Lee's dynamic program [13], and Palermo et al.'s phase search [36]. All three decide the distribution at compile time, but over a cost model rather than by directive or analysis (Section V). The compile-time decision is therefore not the same thing as a closed-form one, and the run-time question is separate from the decision-procedure question.

X. Performance Modeling and Optimization Objectives

Three decision procedures appear in the read set: a closed-form or directive decision, an analytic optimisation over a cost model, and a search over configurations scored by profiles or a simulator (Table I). The procedure determines what the cost model has to represent, and the objectives it optimises.

A. Analytic optimisation in the 1990s

Gupta and Banerjee weight distribution constraints by estimated execution-time penalties, and the authors state that the combined problem is NP-hard and that their estimates rest on compile-time assumptions about loop bounds and branch probabilities [40]. Palermo et al. describe their static pipeline cost model as conservative [36]. Lee's layout choice is a dynamic program over consecutive loops [13]. [TODO: Check how each analytic model in this group represents the network, from the text of [13], [36], [40]; the cards record the assumptions about loop bounds but not yet the network terms.]

B. Volume models in the 2020s

Kong et al. minimise local work plus a weighted communication volume under per-node memory capacity; the cost model is volume-only, with no latency or bandwidth terms, and the paper reports no numeric speedups in its text [41]. GSPMD has no cost model; its sharding is chosen by priority-based propagation [6].

C. Measured and simulated costs from 2019

PipeDream's optimiser uses a single-GPU profile of per-layer compute time, activation size and parameter size, and minimises the time of the slowest stage with a dynamic program over a hierarchical bandwidth model [48]. FlexFlow's simulator profiles each operator once and re-simulates incrementally, and the authors state that it assumes operator times are predictable and independent of input data [47]. Alpa uses profiled stage costs with an integer program for intra-operator choices and dynamic programming for stages [49], and PartIR's automatic tactic uses an analytical simulator with Monte Carlo tree search [7].

D. What the objective shift implies

Finding 2 and Finding 3 in Section VI give the counts. Taken together, the analytic group models the network and the search group models the program's execution. An analytic model needs a network cost accurate enough to rank choices; a profile or simulator needs only the cost of each operator on the target, which can be measured or simulated without a network model [47], [48]. This is the inference behind the shift, and it is testable: a system that uses a network model and a profile on the same workload, compared on the same machine, would separate the two explanations. We did not find a read paper that makes this comparison.

E. Coverage of the objectives

Communication volume is the most common objective across the eras, and local memory fit and productivity grow in the recent eras (Table II). Two notes apply. First, productivity is recorded as an objective in seven of ten Era 6 papers, but none of the read papers measures productivity with a user study; the objective is asserted, not evaluated. Second, the objectives are recorded from the papers' stated goals, not from their measured outcomes.

XI. Multi-GPU, ML, Spatial, and AI-Accelerator Compilers

The recent systems share one design choice: the programmer writes an unpartitioned model and a mesh, and the compiler propagates a layout through the program. The choice is the source of the propagation value on the owner-rule axis (Table IV), and the rest of this section examines what it changes and what it leaves to the programmer.

A. Propagation of shardings

Mesh-TensorFlow expresses a program as named tensor dimensions mapped onto a mesh of processors, so any dimension can be split and the collectives follow from the layouts [32]. GSPMD completes sharding annotations by priority-based propagation and partitions every operator, covering data, model, spatial and pipeline parallelism through one mechanism [6]. PartIR separates partitioning tactics from the model and composes them on an MLIR-based IR, and its propagation uses rewrite rules rather than cost heuristics [7]. Its authors report 50–62% TPUv3 utilisation on up to 2048 cores for GSPMD, without a cost model or a search [6].

Propagation moves the decision from the statements to the values. That makes the compiler's job a constraint problem over the graph, which is heuristic when the constraints are not unique. GSPMD's priority scheme and PartIR's tactic order are two answers to the same ambiguity, and PartIR's authors note that propagation can be blocked by reshapes and non-divisible shapes [7].

B. Search over parallelism plans

FlexFlow searches per-operator configurations with a simulator [47], and Alpa searches intra-operator shardings with an integer program and inter-operator stages with dynamic programming [49]. These systems automate the choice that the propagation systems leave to the programmer, the mesh shape and the stage boundaries. Alpa's authors state that cross-stage communication is not modelled [49], which is the same gap that the analytic group of Section X had for the network.

C. Pipeline and training systems

PipeDream partitions layers into stages from a profile and schedules them with weight stashing [48]; GPipe pipelines micro-batches [58]; Megatron-LM implements tensor parallelism by hand [59]; and ZeRO is catalogued for model-state sharding [60], whose card is pending. We classify the last four as bridge entries because their plan generation is manual or limited.

D. Tensor and sparse-tensor compilers

DISTAL and SpDISTAL bring distribution formats and scheduling commands to dense and sparse tensor algebra, compiled to Legion [8], [45]. Kong et al. derive mappings for tensor DAGs with an SMT solver [41]. Vaidya et al. express distributed sparse tensors as an MLIR dialect that lowers to MPI, with partitions fixed by the programmer [46]. The sparse systems are the only read papers with a sparse-tensor decomposition, and each keeps the partition choice with the user or with a run-time inspector.

E. Spatial and wafer-scale targets

Rocki et al. hand-map a BiCGStab solver onto the Cerebras engine with one mesh column per core and a hardware-routed AllReduce [9]. Stawinoga et al. compile stencil DSLs to the Cerebras software language through three new dialects, with

a runtime library that supports only star-shaped stencils [51]. Jacquelin et al. study scalable distributed high-order stencil computations [61]; the machine and results are pending a read of the paper. Both read wafer-scale papers keep the decomposition static and place one tile per processing element. The compiler's contribution is the communication schedule and the mapping of loops to the fabric, not the choice of decomposition.

[TODO: Add a card for Tiramisu [62], which the catalog lists with distributed-memory support, and for the 2024 Shardy and MLIR documentation once read in full. The accelerator-vendor compilers named in the brief (Graphcore, SambaNova, Tenstorrent, Groq) are not yet catalogued.]

XII. Cross-Era Synthesis

The findings of Section VI and the mechanisms of Sections VII–XI support a short list of generalizations. Table VI states each one with its evidence strength, the observation that would refute it, and what we would need to test it. The list is the contribution of this section; the paper's narrative in earlier sections is the evidence for it.

A. What the generalizations say together

G1 and G2 describe one trade-off from two sides. A run-time stage is needed when ownership is data-dependent, and the cost of choosing a decomposition is paid either by modelling the network or by measuring the program. The systems that avoid the run-time stage pay for it in analysis or in a model, and the systems that pay in measurement avoid the model. G5 and G6 describe the same movement at the level of communication: the unit of reasoning has moved from the statement and the element to the operator and the sharding.

B. What the generalizations do not say

The read set contains no paper from Era 0 or Era 1, so nothing above is a claim about the foundations. The counts are over papers whose full text was read, and the catalog contains 51 further entries whose claims rest on bibliographic records alone. Several generalizations depend on the taxonomy's values, which were coded by one coder; a second coding is a precondition for calling any of them a finding.

C. The cross-era hypothesis about explicit MPI

We hypothesise that explicit MPI won in Eras 3–4 because its cost is transparent to the programmer, whereas high-level partitioning returned in Era 6 because operator graphs make propagation and cost modelling tractable (G2 and G5). The HPF retrospective is the principal evidence for the first half [63]. The second half rests on the propagation designs [6], [7] and is not tested.

[TODO: Test G2 and G5 on the Era 4 PGAS and HPCS papers, which are the least covered era in the read set.]

XIII. Open Problems

Each problem below follows from a generalization in Table VI and names the evidence that would close it.

- **Measured versus modelled cost (G2).** No read paper compares a network model against a profile on one workload. Until one does, the shift from analytic to measured optimisation is an explanation, not a result [40], [47].
- **Unified ownership and layout IR (G1, G7).** Closed-form owners, declared maps and propagated shardings need different analyses. The MLIR shard dialect is the nearest candidate for a shared representation, and the sparse dialect of Vaidya et al. is a second [33], [46].
- **Propagation under constraints.** PartIR's authors report that propagation is blocked by reshapes and non-divisible shapes [7]; GSPMD's sharding completion is heuristic [6]. The conditions under which propagation is unique are not characterised.
- **Overlap of reductions (G6).** Halo exchanges overlap with computation, but among the read papers we did not find a reduction overlapped with the computation that feeds it [9], [54].
- **Break-even of run-time stages.** No read paper reports when an inspector pays for itself against a fully static schedule [34], [43].
- **Correctness of generated communication.** Verification is rarely addressed. The PartIR core is the exception, with a lowering that its authors report as proved [7].
- **Wafer-scale and fine-grained memory.** Both read wafer-scale compilers keep the decomposition static and one tile per processing element, and the stencil runtime supports only star-shaped patterns [9], [51].
- **Productivity.** Productivity is recorded as an objective in most recent papers, but none of the read papers measures it with users [6], [7].

[TODO: Each item needs its test from Table VI run, or an explicit statement that it was not run.]

XIV. Conclusion

Compilers for distributed memory have solved the same three problems in every era: decide who owns each element, decide who computes each operation, and derive the communication between them. What has changed is the decision procedure and the cost the compiler tries to reduce. In the read set, a run-time stage accompanies data-dependent ownership; the cost models moved from analytic networks to measured and simulated programs; the optimisation objective moved from messages to memory and productivity; and the programmer's unit of reasoning moved from the statement to the operator. We state these as generalizations with their evidence strength and their refuting observations (Table VI), and we state the open problems they leave. The taxonomy, the catalog and the comparison tables are generated from the same records, so each count in this paper can be regenerated and challenged. [TODO: Finalise after the second coding and the Era 4 test.]

References

[1] D. Callahan and K. Kennedy, "Compiling programs for distributed-memory multiprocessors," *The Journal of Supercomputing*, 1988.

TABLE VI
GENERALIZATIONS, THEIR EVIDENCE STRENGTH IN THE READ SET, THE OBSERVATION THAT WOULD REFUTE EACH, AND THE TEST THAT WOULD SETTLE IT. STRENGTH IS "SUPPORTED" WHEN THE COUNT IN THE CITED TABLE SHOWS IT WITH NO COUNTEREXAMPLE AMONG THE PAPERS READ, "SUGGESTED" WHEN THE COUNT IS CONSISTENT BUT THE SAMPLE IN ONE ERA IS BELOW TEN, AND "OPEN" WHEN THE CLAIM IS AN INFERENCE WE HAVE NOT TESTED.

ID	Generalization	Strength	Refuting observation	Test
G1	A runtime stage is present whenever the ownership of an element is data-dependent at run time.	supported (17 papers)	A read paper with data-dependent ownership and no run-time stage.	Read the open Vienna Fortran INDIRECT mechanism and any further mapping-array systems.
G2	The cost model moved from analytic (1990s) to profiled or simulated (2019 onward).	suggested	A 2019+ system with an analytic network model as the primary optimiser, or a 1990s system that profiles.	Read the Kali and CHAOS cost discussions in full; read further 2019+ systems.
G3	Message count was the leading objective of the 1990s and has largely given way to memory fit and productivity.	suggested	A recent system that optimises message count as its primary objective.	Re-code objectives on a second axis and compare with the stated goals of each paper.
G4	Block-cyclic and mapping-array decompositions disappear after about 2000, while model-state and sparse partitions appear.	suggested	A post-2005 read system that uses block-cyclic as its main decomposition.	Survey the 2000–2010 PGAS and HPCS papers, which are the least covered era.
G5	Owner-computes is replaced by partition propagation only where the program is a graph of operators.	open	A graph-based system that still uses owner-computes as its main mechanism.	Compare the statement-level rules of Kali and Fortran D with the graph rules of GSPMD on one shared workload.
G6	Element-level communication survives only inside collective systems.	suggested	A read system with element-level generation and no collective component in Era 5 or later.	Read the inspector and element-level systems of 2010 onward.
G7	Halo overlap is the one ownership feature present in every era.	supported (2–3 per era)	A read era with no halo-overlap system.	Extend the read set to Eras 0 and 1.

[2] H. P. Zima, H.-J. Bast, and M. Gerndt, "Superb: A tool for semi-automatic mimd/simd parallelization," *Parallel Computing*, 1988.

[3] M. Chen, Y.-I. Choo, and J. Li, "Compiling parallel programs by optimizing performance," *The Journal of Supercomputing*, 1988.

[4] D. Loveman, "High performance fortran," *IEEE Parallel & Distributed Technology: Systems & Applications*, 1993.

[5] W. Gropp, E. Lusk, N. Doss, and A. Skjellum, "A high-performance, portable implementation of the mpi message passing interface standard," *Parallel Computing*, 1996.

[6] Y. Xu, H. Lee, D. Chen, B. Hechtman, Y. Huang, R. Joshi, M. Krikun, D. Lepikhin, A. Ly, M. Maggioni, R. Pang, N. Shazeer, S. Wang, T. Wang, Y. Wu, and Z. Chen, "Gspmd: General and scalable parallelization for ml computation graphs," arXiv preprint, 2021. [Online]. Available: https://arxiv.org/pdf/2105.04663

[7] S. Alabed, D. Belov, B. Chrzaszcz, J. Franco, D. Grewe, D. Maclaurin, J. Molloy, T. Natan, T. Norman, X. Pan, A. Paszke, N. A. Rink, M. Schaarschmidt, T. Sitdikov, A. Swietlik, D. Vytiniotis, and J. Wee, "Partir: Composing spmd partitioning strategies for machine learning," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025.

[8] R. Yadav, A. Aiken, and F. Kjolstad, "Distal: the distributed tensor algebra compiler," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2022.

[9] K. Rocki, D. V. Essendelft, I. Sharapov, R. Schreiber, M. Morrison, V. Kibardin, A. Portnoy, J. F. Dietiker, M. Syamlal, and M. James, "Fast stencil-code computation on a wafer-scale processor," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.

[10] H. Zima and B. Chapman, "Compiling for distributed-memory systems," *Proceedings of the IEEE*, 1993.

[11] S. Benkner and H. Zima, "Compiling high performance fortran for distributed-memory architectures," *Parallel Computing*, 1999.

[12] D. Unat, A. Dubey, T. Hoefler, J. Shalf, M. Abraham, M. Bianco, B. L. Chamberlain, R. Cledat, H. C. Edwards, H. Finkel, K. Fuerlinger, F. Hannig, E. Jeannot, A. Kamil, J. Keasler, P. H. J. Kelly, V. Leung, H. Ltaief, N. Maruyama, C. J. Newburn, and M. Pericas, "Trends in data locality abstractions for hpc systems," *IEEE Transactions on Parallel and Distributed Systems*, 2017.

[13] P. Lee, "Techniques for compiling programs on distributed memory multicomputers," *Parallel Computing*, 1995.

[14] J. W. Backus, H. Stern, I. Ziller, R. A. Hughes, R. Nutt, R. J. Beeber, S. Best, R. Goldberg, L. M. Haibt, H. L. Herrick, R. A. Nelson, D. Sayre, and P. B. Sheridan, "The fortran automatic coding system," *Papers presented at the February 26-28, 1957, western joint computer conference: Techniques for reliability on - IRE-AIEE-ACM '57 (Western)*, 1957.

[15] L. Lamport, "The parallel execution of do loops," *Communications of the ACM*, 1974.

[16] D. J. Kuck, R. H. Kuhn, D. A. Padua, B. Leasure, and M. Wolfe, "Dependence graphs and compiler optimizations," in *Proceedings of the 8th ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '81*, 1981.

[17] R. Allen and K. Kennedy, "Automatic translation of fortran programs to vector form," *ACM Transactions on Programming Languages and Systems*, 1987.

[18] G. Barnes, R. Brown, M. Kato, D. Kuck, D. Slotnick, and R. Stokes, "The illiac iv computer," *IEEE Transactions on Computers*, 1968.

[19] R. M. Russell, "The cray-1 computer system," *Communications of the ACM*, 1978.

[20] J. B. Dennis and D. P. Misunas, "A preliminary architecture for a basic data-flow processor," *Proceedings of the 2nd annual symposium on Computer architecture - ISCA '75*, 1975.

[21] D. P. Misunas, "Report on the workshop on data flow computer and program organization," MIT Laboratory for Computer Science, Technical Memo MIT/LCS/TM-92, Tech. Rep., 1977, supplied copy; no public record located.

[22] Kung, "Why systolic architectures?" *Computer*, 1982.

[23] C. L. Seitz, "The cosmic cube," *Communications of the ACM*, 1985.

[24] W. D. Hillis and G. L. Steele, "Data parallel algorithms," *Communications of the ACM*, 1986.

[25] H. P. F. Forum, "High performance fortran language specification," *ACM SIGPLAN Fortran Forum*, 1993.

[26] M. Bauer, S. Treichler, E. Slaughter, and A. Aiken, "Legion: Expressing locality and independence with logical regions," in *2012 International Conference for High Performance Computing, Networking, Storage and Analysis*, 2012.

[27] B. Chapman, P. Mehrotra, and H. Zima, "Vienna fortran: A fortran language extension for distributed memory multiprocessors," NASA Contractor Report 187634 / ICASE Report 91-72, Tech. Rep., 1991. [Online]. Available: https://ntrs.nasa.gov/citations/19920002452

[28] P. Mehrotra and J. V. Rosendale, "Programming distributed memory architectures using kali," NASA Contractor Report 187453 / ICASE Report 90-69, Tech. Rep., 1990. [Online]. Available: https://ntrs.nasa.gov/citations/19910002904

[29] C. Koelbel, "Compile-time generation of regular communications patterns," in *Proceedings of the 1991 ACM/IEEE conference on Supercomputing - Supercomputing '91*, 1991.

[30] S. Chatterjee, J. R. Gilbert, F. J. E. Long, R. Schreiber, and S.-H. Teng, "Generating local addresses and communication sets for data-parallel programs," in *Proceedings of the fourth ACM SIGPLAN symposium on Principles and practice of parallel programming - PPOPP '93*, 1993.

[31] U. Bondhugula, "Compiling affine loop nests for distributed-memory parallel architectures," in *Proceedings of the International Conference*

on High Performance Computing, Networking, Storage and Analysis, 2013.

[32] N. Shazeer, Y. Cheng, N. Parmar, D. Tran, A. Vaswani, P. Koanantakool, P. Hawkins, H. Lee, M. Hong, C. Young, R. Sepassi, and B. Hechtman, "Mesh-tensorflow: Deep learning for supercomputers," arXiv preprint (also NeurIPS 2018), 2018. [Online]. Available: https://arxiv.org/pdf/1811.02084

[33] L. community, "Mlir 'shard' dialect (official documentation; formerly 'mesh')," MLIR documentation (mlir.llvm.org), 2025. [Online]. Available: https://mlir.llvm.org/docs/Dialects/Shard/

[34] J. Saltz, K. Crowley, R. Michandaney, and H. Berryman, "Run-time scheduling and execution of loops on message passing machines," *Journal of Parallel and Distributed Computing*, 1990.

[35] S. Ramaswamy, B. Simons, and P. Banerjee, "Optimizations for efficient array redistribution on distributed memory multicomputers," *Journal of Parallel and Distributed Computing*, 1996.

[36] D. J. Palermo, E. W. H. IV, and P. Banerjee, "Dynamic data partitioning for distributed-memory multicomputers," *Journal of Parallel and Distributed Computing*, 1996.

[37] R. Das, R. Ponnusamy, J. Saltz, and D. Mavriplis, "Distributed memory compiler methods for irregular problems: data copy reuse and runtime partitioning," NASA Contractor Report 187635 / ICASE Report 91-73, Tech. Rep., 1991. [Online]. Available: https://ntrs.nasa.gov/citations/19920002451

[38] Y. Hwang, B. Moon, S. D. Sharma, R. Ponnusamy, R. Das, and J. H. Saltz, "Runtime and language support for compiling adaptive irregular programs on distributed-memory machines," *Software: Practice and Experience*, 1995.

[39] A. Rogers and K. Pingali, "Compiling for distributed memory architectures," *IEEE Transactions on Parallel and Distributed Systems*, 1994.

[40] M. Gupta and P. Banerjee, "Demonstration of automatic data partitioning techniques for parallelizing compilers on multicomputers," *IEEE Transactions on Parallel and Distributed Systems*, 1992.

[41] M. Kong, R. A. Yosef, A. Rountev, and P. Sadayappan, "Automatic generation of distributed-memory mappings for tensor computations," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023.

[42] J. Saltz, H. Berryman, and J. Wu, "Multiprocessors and run-time compilation," *Concurrency: Practice and Experience*, 1991.

[43] R. Ponnusamy, J. Saltz, A. Choudhary, Y.-S. Hwang, and G. Fox, "Runtime support and compilation methods for user-specified irregular data distributions," *IEEE Transactions on Parallel and Distributed Systems*, 1995.

[44] E. Slaughter, W. Lee, S. Treichler, M. Bauer, and A. Aiken, "Regent: a high-productivity programming language for hpc with logical regions," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2015.

[45] R. Yadav, A. Aiken, and F. Kjolstad, "Spdistal: Compiling distributed sparse tensor computations," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.

[46] M. Vaidya, S. Singh, D. Mantri, M. S. Eydenberg, B. M. Kelley, S. Rajamanickam, A. Rountev, and P. Sadayappan, "Distributed sparse tensor computations in mlir," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025.

[47] Z. Jia, M. Zaharia, and A. Aiken, "Beyond data and model parallelism for deep neural networks," Proceedings of Machine Learning and Systems (MLSys) 1, 2019. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/hash/b422680f3db0986ddd7f8f126baaf0fa-Abstract.html

[48] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, "Pipedream: generalized pipeline parallelism for dnn training," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019.

[49] L. Zheng, Z. Li, H. Zhang, Y. Zhuang, Z. Chen, Y. Huang, Y. Wang, Y. Xu, D. Zhuo, E. P. Xing, J. E. Gonzalez, and I. Stoica, "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), 2022. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin

[50] S. Hiranandani, K. Kennedy, and C.-W. Tseng, "Evaluation of compiler optimizations for fortran d on mimd distributed memory machines," in *Proceedings of the 6th international conference on Supercomputing - ICS '92*, 1992.

[51] N. Stawinoga, D. Katz, A. Lydike, J. Zarins, N. Brown, G. Bisbas, and T. Grosser, "An mlir lowering pipeline for stencils at wafer-scale," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '26)*, 2026.

[52] B. Chapman, P. Mehrotra, and H. Zima, "Programming in vienna fortran," *Scientific Programming*, 1992.

[53] S. Hiranandani, K. Kennedy, and C.-W. Tseng, "Compiler optimizations for fortran d on mimd distributed-memory machines," in *Proceedings of the 1991 ACM/IEEE conference on Supercomputing - Supercomputing '91*, 1991.

[54] G. Bisbas, A. Lydike, E. Bauer, N. Brown, M. Fehr, L. Mitchell, G. Rodriguez-Canal, M. Jamieson, P. H. J. Kelly, M. Steuwer, and T. Grosser, "A shared compilation stack for distributed-memory parallelism in stencil dsls," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024.

[55] Y. N. Shen, R. C. Zhao, and L. Wang, "Eliminating redundant communication of code generation for distributed memory machines," in *Eighth ACIS International Conference on Software Engineering, Artificial Intelligence, Networking, and Parallel/Distributed Computing (SNPD 2007)*, 2007.

[56] U. Bondhugula, "Automatic distributed-memory parallelization and code generation using the polyhedral framework," IISc-CSA-TR-2011-3, Indian Institute of Science, Tech. Rep., 2011. [Online]. Available: https://www.csa.iisc.ac.in/~udayb/publications/uday11distmem-tr.pdf

[57] R. Dathathri, "Compiling for a dataflow runtime on distributed-memory parallel architectures," Master's thesis, Indian Institute of Science, 2014, m.Sc. (Engineering) thesis.

[58] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," arXiv preprint (also NeurIPS 2019), 2018. [Online]. Available: https://arxiv.org/pdf/1811.06965

[59] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," arXiv preprint, 2019. [Online]. Available: https://arxiv.org/pdf/1909.08053

[60] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.

[61] M. Jacquelin, M. Araya–Polo, and J. Meng, "Scalable distributed high-order stencil computations," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.

[62] R. Baghdadi, J. Ray, M. B. Romdhane, E. D. Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. Amarasinghe, "Tiramisu: A polyhedral compiler for expressing fast and portable code," in *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2019.

[63] K. Kennedy, C. Koelbel, and H. Zima, "The rise and fall of high performance fortran," in *Proceedings of the third ACM SIGPLAN conference on History of programming languages*, 2007.